

CSE4067 Introduction to Blockchain Programming: Assignment

goshgar.ismayilov@boun.edu.tr

October 2024

Part I: Preparation (+0)

Install **Metamask** extension to your browser.

- <https://metamask.io/>

Choose the test network on your **Metamask** wallet.

- **Ethereum Sepolia**

Find a corresponding **faucet** and get some free tokens.

- e.g. <https://cloud.google.com/application/web3/faucet/ethereum/sepolia>

Set up a new project on **Remix**.

- <https://remix.ethereum.org/>

Part II: Smart Contract (+50)

Implement an autonomous decentralized lottery smart contract (i.e. **MU-LOT**) where each user makes payment to purchase a single lottery ticket to participate and only one user eventually wins. The winner user is determined by combining the random numbers of all users. The lottery consists of two stages as:

- **Purchase**: Purchase a lottery ticket to participate in that lottery session
- **Decide**: Decide the lottery winner and claim the prize

Purchase and **Decide** stages have certain time-based durations, (e.g. 1 hour). The durations will be initially set by the contract deployer (i.e. contract owner).

Each user must supply hash of a random number in **Purchase** stage to get ticket. This stage is required so that nobody can initially see the random numbers of the other users. No user can purchase a ticket once **Purchase** stage is over.

Each user must pay exactly the ticket fee to participate in **Purchase** stage. The excess must be refunded back. The sum of all these fees is the prize of that

lottery. Ticket fee is dynamically adjusted by using an external oracle contract, which is already deployed on Sepolia.

- `TicketFeeOracle` at address `0xAD39623a8Cd97185755310Cec8AFDb19Fe330D5A`
- make external-call to `TicketFeeOracle.getTicketFee()` function

Once **Purchase** stage is over, **Decide** stage must begin where the users now reveal their random numbers. The correctness of the random numbers revealed must be checked. In case a user does not reveal on time, that user cannot win lottery any more by forfeiting payment.

The random numbers from all the users in **Purchase** stage must be combined (e.g. **XORed**) to get the final random number. The mechanism to get the final random number is defined in the following website:

- <https://ethereum.stackexchange.com/questions/191/how-can-i-securely-generate-a-%20random-%20number-in-my-smart-contract>

The final random number must determine the lottery winner. All the users can check whether they win the lottery or not. However, only the winner user can successfully claim and win it. The winner can claim it at any time once **Decide** stage ends. When the total prize is transferred to winner, the contract must self-destruct itself.

Part III: Web Interface (+25)

Build a simple web user interface for Part II that connects to your **Metamask** wallet. The interface must allow to deploy a new lottery contract to the testnet.

- The development platform for the interface is upon to your decision.
- **JavaScript** is recommended to be able to use `web3.js` or `ethers.js`.
- <https://medium.com/coinmonks/solidity-and-web3-js-141115b0f8c5>

Part IV: Zero-Knowledge Proof (+15)

Zero-knowledge proof is a protocol where prover proves statement to verifier without disclosing that statement itself.

- I can prove that I know the cube-root of **X** without revealing the answer.
- **ZoKrates** framework helps proof generation and verification.
- <https://zokrates.github.io>

Activate **ZoKrates** in **Remix**.

Use **Code-1** to prove cube-root statement in **main.zok**.

Compile **main.zok** in **Remix**.

Listing 1 Code-1

```
1  def main(private field a, field b) {  
2      assert(a * a * a == b);  
3      return;  
4  }
```

Run setup `main.zok` in Remix.

Export and compile `verifier.sol` contract in Remix.

Deploy **Verifier** - `verifier.sol` to Sepolia. (Not Pairing - `verifier.sol`.)

Compute a witness for `a=1234` and `b=1879080904` in Remix. Note that `a` is private data that you only know and `b` is public data that everyone sees.

Generate and copy proof for your cube-root statement in Remix.

Verify that proof on the deployed `verifier.sol` contract with `verifyTx` function. You should see `true` as the output.

Part V: Documentation and Comments (+10)

Prepare comprehensive report to explain system design and gas consumption of smart contract functions in table. Quality of comments will be also evaluated.

The report must contain the address(es) of the smart contract(s) deployed on the testnet. Ensure that the contract is successfully deployed before due date and always externally accessible.

This assignment can be done in groups of 1 or 2 students, with **5-week** long after its announcement. Cheating (e.g. high similarity between assignments, help from AI tools) will be penalized. Submit your assignment with the following format. Single submission by a student for 2-Student groups is sufficient.

- 1 Student: `#No1.zip`
- 2 Students: `#No1_#No2.zip`

Timestamp your assignment before you submit it. Keep the assignment files and the resulting `.ots` file(s).

- <https://opentimestamps.org/>